

COMPUTERRL: 扩展面向计算机使用智能体的端到端在线强化学习

Hanyu Lai^{1*†}, Xiao Liu^{1,2*}, Yanxiao Zhao^{3*†}, Han Xu², Hanchen Zhang^{1†}, Bohao Jing^{2†}, Yanyu Ren^{1†}, Shuntian Yao^{1†}, Yuxiao Dong¹, Jie Tang¹

¹ 清华大学 ² 智谱 AI ³ 中国科学院大学

摘要

我们提出 COMPUTERRL，这是一套面向自主桌面智能的框架，使智能体能够熟练操作复杂的数字工作空间。COMPUTERRL 采用 API-GUI 范式，将程序化 API 调用与直接 GUI 交互统一起来，以解决机器智能体与以人为中心的桌面环境之间固有的不匹配问题。扩展端到端强化学习 (RL) 训练，对于提升智能体能力以及在多样化桌面任务上的泛化能力至关重要；然而，由于环境效率低下，以及长时间训练过程中的不稳定性，这一目标仍然难以实现。为了支持可扩展且稳健的训练，我们开发了一套分布式 RL 基础设施，可编排数千个并行虚拟桌面环境，从而加速大规模在线 RL。此外，我们提出 Entropulse，这是一种在强化学习与监督微调之间交替进行的训练策略，可有效缓解长时间训练过程中的熵坍塌。我们将 COMPUTERRL 应用于开放模型 GLM-4-9B-0414 与 GLM-4.1V-9B-Thinking，并在 OSWorld 基准上进行评测。AutoGLM-OS-9B 达到了新的最先进准确率 **48.9%**，表明通用智能体在桌面自动化方面取得了显著提升。我们的代码与新基准 OfficeWorld 已发布于我们的 GitHub 仓库。该算法与框架也被用于构建 AutoGLM (Liu et al., 2024b)。

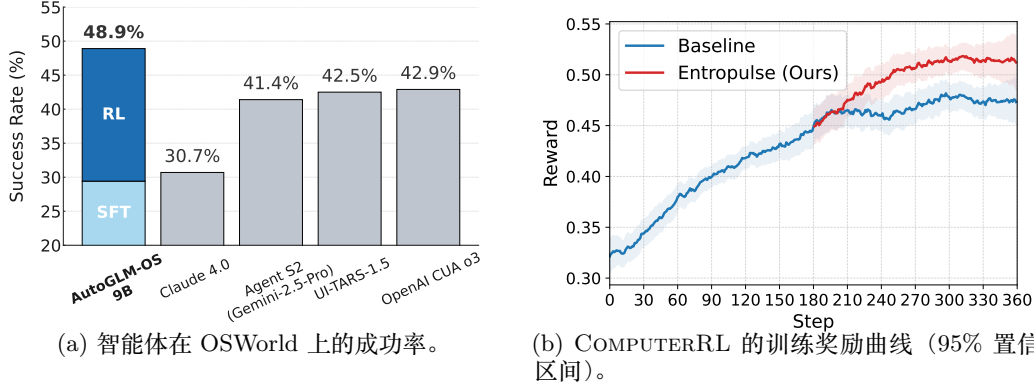


图 1: COMPUTERRL 为操作系统智能体实现了高效的端到端在线策略优化。(a) 在 OSWorld (Xie et al., 2024) 上，使用 COMPUTERRL 训练的 AutoGLM-OS 超越了最先进的智能体。(b) 我们的 Entropulse 方法获得了更高的平均训练奖励，并且相较传统方法同时提升了学习效率与最终性能。

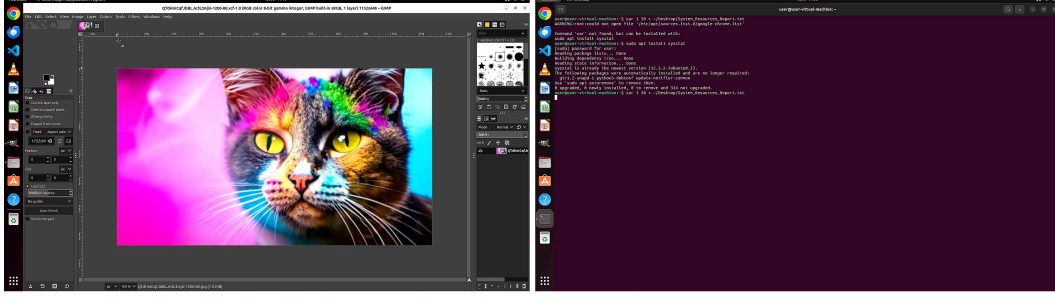
1 引言

大语言模型 (LLM) (Achiam et al., 2023; Touvron et al., 2023b; Zeng et al., 2022; GLM et al., 2024; Team et al., 2023; Guo et al., 2025; Bai et al., 2023a; Yang et al., 2025a) 极大拓展了人工智能能力的广度与深度，推动人们重新深入审视对机器智能的认识。在所有应用场景中，基于 LLM 的 GUI (图形用户界面) 智能体尤其受到研究者关注 (Xi et al., 2023; Wang et al., 2023; Liu et al., 2023); 这类智能体能够在用户设备上独立完成感知、推理与复

*HL、XL 与 YZ 贡献相同。

†这些作者在智谱 AI 实习期间完成本职工作。

(A): Copy an image from a .xcf file on the Desktop and paste it into a LibreOffice Writer document, then save it as 'image.docx' on the Desktop. (B): Use the 'sar' command from the 'sysstat' toolkit to monitor Ubuntu sys resources every second for 30 seconds, and save the results to 'System_Resources_Report.txt' on the Desktop.



(C): Create a table with 'Month' and 'Total' headers in a new sheet to show the total sales for all months. (D): Convert all uppercase text to lowercase in a document for consistency.

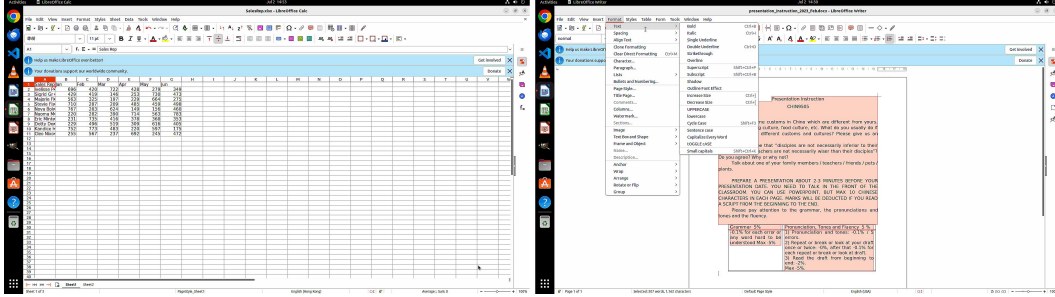


图 2: AUTOGLM-OS 执行四项用户任务的示例，包括在 GIMP 与 LibreOffice Writer 之间处理图像、在终端中监控系统资源使用情况、在 LibreOffice Calc 中进行表格计算，以及在 LibreOffice Writer 中设置文档格式。

杂任务执行。鉴于桌面环境仍是智能密集型任务的核心载体，开发计算机使用智能体对于从根本上改变人机交互方式并提升 AI 能力至关重要 (Agashe et al., 2025; Wu et al., 2024a)。

尽管此前已有多项开发计算机使用智能体的尝试 (Agashe et al., 2025; Lei et al., 2024; Xie et al., 2025)，使其在真实场景中长时间自主运行仍是一项重大挑战。第一个主要障碍源于 GUI 在本质上是为人交互设计的，因此，让 GUI 智能体 (Liu et al., 2024c; OpenAI, 2025; Qin et al., 2025) 模拟人类动作并非易事，而且过程繁琐。第二，当前主流的行为克隆 (BC) (Bain & Sammut, 1995; Bratko et al., 1995) 方法，包括人工标注 (He et al., 2025) 与模型蒸馏 (Sun et al., 2024; Xu et al., 2024)，在可扩展性与有效性方面均受到限制。人工标注虽然精确，但面对复杂任务时需要高得难以承受的人力成本。另一方面，模型蒸馏受限于教师模型的性能，因而限制了整体能力。这两类方法通常都表现出较差的泛化能力与有限的错误恢复能力。最后，尽管强化学习 (RL) 已在桌面自动化任务中展现潜力 (Lu et al., 2025; Feng et al., 2025)，其实际应用仍受计算复杂度与方法挑战所限。复杂环境、缓慢收敛以及 RL 训练中已知的低效率问题 (Xie et al., 2024; Bonatti et al., 2024; Yu et al., 2025; Fu et al., 2025)，严重阻碍了其在计算机使用智能体训练中的大规模采用。

在本工作中，我们提出 COMPUTERL，这是一套端到端算法框架，旨在推进桌面级规划、推理与设备操作。该框架包含一种新的 API-GUI 交互范式、一套面向计算机环境的可扩展 RL 训练基础设施，以及一种能够延长有效训练过程的 RL 算法。首先，我们提出 API-GUI：一个大规模、自动构建的 API 生态系统，使智能体能够超越面向人类的操作范式所带来的固有偏置。它采用更面向机器的设备交互方式，将 API 调用与 GUI 动作结合，从而显著提升智能体的通用性与整体性能。其次，我们开发了一套基于 Docker 和 gRPC 协议的虚拟机集群分布式训练基础设施，以实现可扩展性，并与 AgentBench (Liu et al., 2023) 完全兼容。该基础设施支持数千个并行环境，确保系统具备高度可扩展性，并使各环境的交互保持一致。此外，我们将训练基础设施与 AgentRL 框架 (Zhang et al., 2025) 集成，以支持高效的异步训练，从而加速训练过程。最后，为应对 RL 训练中的停滞与收敛问题，尤其是熵坍塌和 KL 散度上升，我们提出 Entropulse，周期性地地在 RL 阶段与 SFT 阶段之间交替。该方法能够维持探索能力，并确保性能持续提升 (图 1b)。

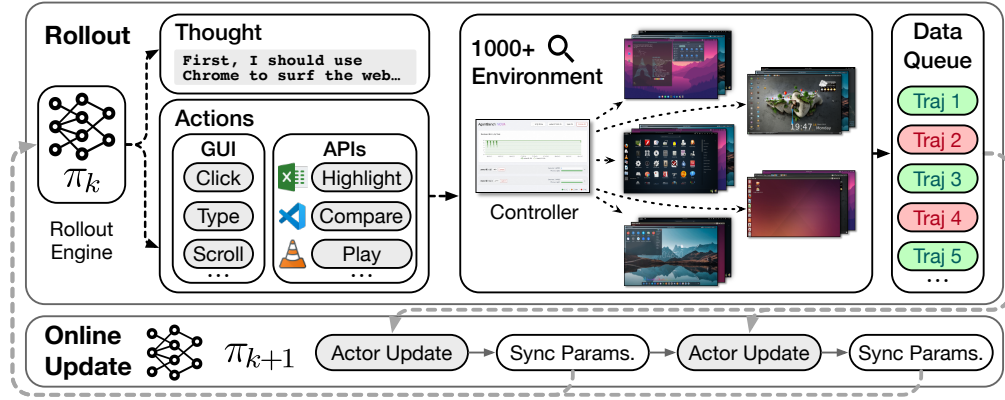


图 3: COMPUTERRL 框架概览。我们提出一种 API-GUI 动作范式，将自动构建的 API 与 GUI 动作无缝结合，以提高智能体的效率与有效性。包含 1,000 多个真实实例的大规模并行桌面环境与异步 RL 框架相结合，可实现高效采样与稳健的智能体训练。

因此，通过在桌面环境中利用端到端 RL 与优化，COMPUTERRL 在理解和操作 GUI 方面取得了显著进步。OSWorld 基准 (Xie et al., 2024) 上的评测表明，COMPUTERRL 在计算机使用挑战中取得显著提升 (见图 1a)，成功率达到 **48.9%** (RL 带来 66% 的性能增益)，超过 OpenAI CUA o3 (42.9%)、UI-TARS-1.5 (42.5%) 和 Anthropic Claude Sonnet 4 (30.7%) 等其他最先进模型。

概括而言，我们的贡献如下：

- 我们提出一种新的交互范式：通过引入与传统 GUI 操作相结合的大规模自动构建 API 生态系统，使交互从以人为中心转向面向机器。该方法解决了人为设计的界面与人工智能能力之间固有的不匹配，同时在基于计算机的任务上实现了更高的操作效率与泛化性能。
- 我们通过重构虚拟机集群，为计算机使用智能体建立了一套大规模分布式 RL 基础设施；它凭借数千个并行环境和与 AgentBench 的无缝兼容，达到了前所未有的可扩展性，从而克服了长期限制基于 RL 的计算机使用智能体训练扩展实验规模的关键瓶颈，并使大规模智能体训练取得突破性结果成为可能。
- 我们提出 Entropulse，这是一种新的训练方法，通过策略性地交替 RL 与 SFT 阶段，系统解决长时间 RL 训练中的熵坍塌与 KL 散度累积问题，从而实现持续的性能提升，并在计算机自动化任务中达到最先进性能。

2 COMPUTERRL 框架

GUI 面向人类的设计会阻碍智能体效率，而环境可扩展性有限又制约了大规模训练。本节介绍 COMPUTERRL 框架 (见图 3)。该框架采用 API-GUI 范式，将类人的 GUI 交互与高效 API 调用相结合。此外，我们开发了一个可扩展、可并行运行的 Ubuntu 桌面环境，并采用全异步 RL 框架来实现高效训练。

2.1 通用 API-GUI 范式

现有 GUI 智能体由于依赖类人交互而面临挑战；基于 API 的控制虽然高效，却会引入实现复杂性与安全限制。为解决这些问题，我们提出 API-GUI 范式，将两类动作空间统一起来，使智能体既能利用 API 的效率，又能保留 GUI 的通用性。

我们开发了一套基于 LLM 的应用程序 API 自动开发工作流 (Yang et al., 2024a; Wang et al., 2024)，显著降低了创建 API 的门槛。用户提供示例任务，系统则通过以下三个阶段自主生成 API 代码与测试用例：

- 需求分析：用户为目标应用程序提供任务示例。我们的 LLM 分析这些实例，提取必要功能，并与现有 API 接口对比以识别缺口。系统会为尚未覆盖的功能自动生成新接口，并优先设计通用函数，以降低复杂度并增强易用性。

- **API 实现：**该 workflow 逐个遍历接口定义，使用指定的 Python 库实现 API 功能。系统还会实现错误处理机制与日志记录，以便调试和维护。
- **测试用例生成：**与 Li & Yuan (2024) 类似，我们从两方面验证 API 的正确性：(1) 调用时不存在运行时错误；(2) 在不同参数输入下返回正确结果。失败的 API 会接收错误反馈并进行自主修正。

该方法只需极少的人工干预即可创建面向特定应用程序的 API。我们已经为多个 Ubuntu 应用程序开发了 API 集，并通过实验验证了其有效性。详细的 API 开发 workflow 见附录 A。智能体动作空间与提示词设计分别详见附录 B 和 C。

2.2 面向大规模并行的稳定 UBUNTU 环境

稳定且可扩展的 Ubuntu 环境，是构建行为克隆数据并开展大规模 RL 训练的必要条件。在 OSWorld (Xie et al., 2024) 基础上，我们识别出以下关键局限：

- **资源密集与稳定性问题：**虚拟机大量占用 CPU，并且在高并发下不稳定，会造成性能下降与系统卡死。
- **网络瓶颈：**繁重工作负载会带来网络开销、连接失败与 IP 地址丢失，妨碍智能体交互和日志记录。
- **缺少原生分布式支持：**OSWorld 不支持多节点集群，因而无法高效进行分布式部署。

为解决这些局限，我们构建了一套稳健且可并行化的 OSWorld 基础设施（见图 3），包含以下创新：

- **标准化的解耦接口：**我们通过 AgentBench API 重构环境，提供统一接口，将环境执行与计算后端解耦，并支持灵活的资源管理。
- **轻量级虚拟机部署：**我们使用 qemu-in-dockere 部署容器化 Ubuntu 虚拟机；精简后的镜像减少了网络问题并优化资源使用，显著降低每个实例的 CPU 消耗。
- **分布式多节点集群：**我们采用基于 gRPC 的通信，将多个 CPU 节点连接为分布式集群，并集中完成资源分配与编排。
- **基于 Web 的可视化与监控：**Web 界面可实时展示环境状态、智能体状态和资源分配情况，从而提升易用性与调试能力。

凭借这些技术改进，我们的系统能够在多节点 CPU 集群上部署数千个并发环境，这一点已经得到大量实证评估的验证。结果证实，我们的平台具有出色的稳定性、资源效率与可扩展性，可作为大规模 RL 与智能体研究的基础设施。

2.3 用于高效训练的全异步 RL 框架

现有 RL 框架依赖同步训练范式，交替进行轨迹采集与参数更新，导致训练效率低下。为解决这一局限，我们使用 AgentRL 框架 (Zhang et al., 2025) 开展全异步 RL 训练，并采用以下设计：

- **资源分区：**数据采集在专用资源上运行，训练器则从回放引擎以流式方式获取数据，从而避免二者相互阻塞。
- **动态批大小：**训练器以灵活的批大小处理传入数据，减少空闲时间并提高效率。
- **模块化组件隔离：**Actor、参考模型和 Critic 模块分别在专用资源上独立运行。我们使用 PyTorch 分布式进程组与 NCCL 高效共享参数。
- **缓解离策略偏差：**我们限制回放缓冲区大小，并在每次更新后同步轨迹，确保轨迹与最新策略保持接近。

通过稳定的高并发桌面环境，以及将训练与 rollout 解耦，我们显著提高了采样和 RL 训练效率。系统实现了较高的单 GPU 平均功耗，反映出较优的资源利用率。该设计通过动态平衡工作负载来支持可扩展、高吞吐的 RL 训练，从而显著提升硬件效率与整体训练吞吐量。

3 COMPUTERRL 的训练方法

在第 2 节中，我们为大规模智能体训练建立了稳健基础。然而，扩展端到端训练仍面临两项挑战：如何初始化一个具备能力的基础策略，以及 RL 期间出现的熵坍塌。本节详述可扩展的 COMPUTERRL 训练方法，以及为延长桌面环境中的有效训练过程而提出的算法创新。

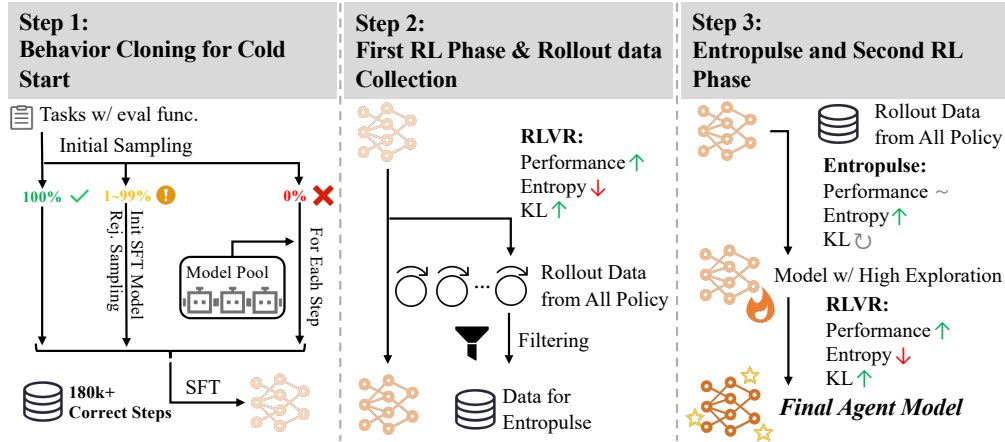


图 4: COMPUTERRL 概览, 包含三个阶段: (1) 使用通用 LLM 收集的轨迹进行 BC 冷启动; (2) 使用可验证、基于规则的奖励进行步骤级 GRPO 强化学习; (3) Entropulse, 在 RL 与基于正确 rollout 的 SFT 之间交替, 以恢复熵并维持学习。

3.1 行为克隆设置

为了让模型实现冷启动, 我们将行为克隆 (BC) 作为训练的初始阶段。BC 通过模仿用户交互, 使智能体获得基础能力, 进而能够快速适应计算机操作与任务。

使用多个 LLM 收集轨迹。我们人工收集了大量任务及其对应的评估函数 (见附录 E), 并通过扩增构建了包含 8,000 个任务的数据集。然而, 大规模收集高质量轨迹仍然困难。人工标注成本高得难以承受, 而仅依赖单一模型生成轨迹, 则会受到该模型能力限制, 导致数据分布有限且同质化。为解决这些局限, 我们利用多个先进模型的互补优势, 收集多样且高质量的交互轨迹。具体而言, 我们的数据流水线包含三个关键阶段:

1. **初始采样**: 对于每项任务, 我们使用闭源 LLM, 相互独立地为每项任务采样若干条轨迹。我们既记录完整的交互轨迹, 也记录相应评估函数的输出。该过程得到一组丰富而多样的轨迹, 为后续数据扩增与模型适配奠定基础。
2. **结果分层**: 完成初始数据收集后, 我们依据达到的准确率对任务结果进行分层分析, 将所有任务分为三组: 完全解决 ($\text{acc} = 100\%$)、部分解决 ($0 < \text{acc} < 100\%$) 和未解决 ($\text{acc} = 0\%$)。
3. **采用分层采样的任务导向扩增**: 对于部分解决的任务, 我们使用初始轨迹作为输入, 对骨干模型进行 SFT。随后使用微调后的模型为每项任务采样更多轨迹, 从而大幅扩展模型此前能力有限的任务所对应轨迹的覆盖范围与质量。

对于被归类为未解决的任务, 我们构建一个高性能模型池, 并随机选择其中一个模型来决定每个动作。该方法利用了不同模型在任务层面的差异: 即使总体性能相近, 各模型仍具有不同的专长领域, 因此能够生成任何单一模型都无法独立得到的轨迹。

我们系统地汇总并筛选所收集的交互数据, 只保留成功轨迹 (超过 18 万个正确步骤), 并用它们对模型进行监督微调。这一策略使模型获得稳健的桌面操作能力与基础推理能力, 显著提升基础模型的性能。

3.2 采用可验证奖励的强化学习

步骤级组相对策略优化。我们将 GRPO 算法 (Shao et al., 2024) 扩展到步骤级, 使其更适合智能体 RL 训练。对于每个任务 τ , 策略 π_θ 与桌面环境交互, 并采样 G 条轨迹 $\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_G$ 。第 i 条轨迹由 L_i 个步骤级动作 $o_{i,1}, \dots, o_{i,L_i}$ 组成。来自同一任务的所有步骤被归为一组,

并为每一步计算优势 $A_{i,j}$ 。总损失按如下方式汇总所有步骤的优势：

$$\mathcal{J}_{StepGRPO}(\theta) = \mathbb{E}_{\mathcal{T} \sim P(\mathcal{T}), \{\{o_{i,j}\}_{j=1}^{L_i}\}_{i=1}^G \sim \pi_{\theta_{old}}} \left[\frac{1}{\sum_{i=1}^G L_i} \sum_{i=1}^G \sum_{j=1}^{L_i} \left(\min \left(\frac{\pi_{\theta}(o_{i,j}|q_{i,j})}{\pi_{\theta_{old}}(o_{i,j}|q_{i,j})} A_{i,j}, \right. \right. \right. \\ \left. \left. \left. \text{clip} \left(\frac{\pi_{\theta}(o_{i,j}|q_{i,j})}{\pi_{\theta_{old}}(o_{i,j}|q_{i,j})}, 1 - \epsilon, 1 + \epsilon \right) A_{i,j} \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} \parallel \pi_{ref}) \right) \right],$$

$$A_{i,j} = \frac{r_{i,j} - \text{mean}(\mathcal{R})}{\text{std}(\mathcal{R})}, \quad \mathcal{R} = \{r_{u,v} \mid u = 1, \dots, G, v = 1, \dots, L_u\}$$

奖励设计。我们从构建的人工标注数据（见第 3.1 节）中选取一个子集用于 RL，并采用基于规则的验证函数，为每条轨迹提供可验证的训练信号。对于成功解决任务的轨迹，每个格式正确且对解决任务有贡献的动作都获得奖励 1；失败轨迹或格式不正确的动作获得奖励 0。不同于通过贝尔曼方程传播逐步回报的传统方法，我们的方法将每一组提示—响应视为独立训练实例，并根据最终轨迹结果赋予奖励。通过将智能体行为与任务成功直接耦合，这种直接奖励分配方式提供了明确反馈，从而促进有效的策略优化。

3.3 使用 ENTROPULSE 扩展 RL 训练

在第 3.2 节的 RL 训练中，我们观察到，经过数百个训练步骤后模型性能便进入平台期，任务完成率停滞且熵持续下降。这种过早收敛促使我们研究如何延长有效训练并增强策略探索。受 DAPO (Yu et al., 2025) 启发，我们尝试提高裁剪阈值；这一做法减缓了熵的下降，却也显著降低了策略改进速度。

为解决这一问题，我们提出 Entropulse。其动机来自我们的观察：SFT 与 RL 的目标在训练期间存在显著差异。随着 RL 优化过程中熵逐渐下降，在关键节点引入 SFT 可以增强探索与轨迹多样性，从而促进进一步的策略优化。在初始 RL 训练期间，我们汇总并保留所有成功的 rollout 轨迹。按照惯例，这些轨迹使用一次后便会被丢弃；然而，它们来自不同训练步骤中的不同策略，是有价值且多样的行为数据。

我们处理该数据集时，会针对每个不同任务随机选择成功轨迹，构建新的 SFT 训练集。该训练集具有以下属性：

1. 高质量：所有数据均由完整、高保真的轨迹组成。
2. 多样性：Rollout 来自不同训练步骤中的异构策略，因而包含多种问题求解策略。
3. 计算高效：该数据集利用已有交互数据，无需额外进行环境 rollout。

在这一精选数据集上进行 SFT，会使策略行为发生显著变化。虽然评测任务性能保持稳定，但得到的策略相较原始策略具有更高的熵，表明其探索能力得到增强。在这一增强的探索能力之上，我们开展第二轮 RL 训练，取得了显著的性能提升，并最终在计算机自动化任务中达到最先进水平。训练细节见附录 D。

4 实验

我们将 COMPUTERRL 应用于 GLM-4-9B-0414 (GLM et al., 2024) 与 GLM-4.1V-9B-Thinking (Hong et al., 2025)，得到 AUTOGLM-OS-9B。我们在多种场景下开展广泛实验，以评估 AUTOGLM-OS 在计算机环境中的性能。

4.1 主要结果

为了更贴近真实用户体验，我们在 OSWorld (Xie et al., 2024) 和 OSWorld-Verified 基准上评估 AUTOGLM-OS，并将其性能与 CUA (OpenAI, 2025)、Claude-4 (Anthropic, 2023) 和 UI-TARS (Qin et al., 2025) 等最先进模型进行比较。比较结果见表 1。结果表明，AUTOGLM-OS 在多个领域均取得了更优性能，其中优势在具有挑战性的多应用设置中最为明显。此外，借助 API-GUI 策略，AUTOGLM-OS 最多只需最强基线方法所用步骤的 1/3 即可完成，展现出显著的执行效率提升。这些结果凸显了 COMPUTERRL 在多种应用中推进计算机自动化技术前沿的潜力。

表 1: AutoGLM-OS 在 OSWorld 与 OSWorld-Verified 上的性能（更新于 2025 年 8 月）。我们将 AutoGLM-OS 与包括专有模型和开放模型在内的最先进智能体进行比较。

智能体模型	参数量	OSWorld	OSWorld-Verified
专有模型			
Aria-UI w/ GPT-4o (Yang et al., 2024b)	-	15.2	-
Aguvis-72B w/ GPT-4o (Xu et al., 2024)	-	17.0	-
Claude 3.7 Sonnet (Anthropic, 2023)	-	28.0	35.8
Claude 4.0 Sonnet (Anthropic, 2023)	-	30.7	43.9
Agent S2 w/ Claude-3.7-Sonnet (Agashe et al., 2025)	-	34.5	-
InfantAgent (Lei et al., 2024)	-	35.3	-
OpenAI CUA 4o (OpenAI, 2025)	-	38.1	31.3
Agent S2 w/ Gemini-2.5-Pro (Agashe et al., 2025)	-	41.4	45.8
UI-TARS-1.5 (Qin et al., 2025)	-	42.5	-
OpenAI CUA o3 (OpenAI, 2025)	-	42.9	-
开放模型			
Qwen2.5-vl-72B (Bai et al., 2023b)	72B	8.8	5.0
PC Agent-E (He et al., 2025)	72B	14.9	-
UI-TARS-72B-SFT (Qin et al., 2025)	72B	18.8	-
UI-TARS-72B-DPO (Qin et al., 2025)	72B	24.6	27.1
UI-TARS-1.5-7B (Qin et al., 2025)	7B	26.9	27.4
Jedi-7B w/ GPT-4o (Xie et al., 2025)	7B+	27.0	29.3
UI-TARS-7B-1.5 + ARPO (Lu et al., 2025)	7B	29.9	-
COMPUTERRL (本文)			
w/ GLM-4-9B-0414	9B	<u>48.1±1.0</u>	<u>47.3</u>
w/ GLM-4.1V-9B-Thinking	9B	48.9±0.5	48.0

4.2 办公应用性能

办公应用是交付与展示工作成果的关键界面，因此构成了评估计算机使用智能体的重要测试平台。为了评估智能体在该领域的性能，我们从三个来源精选了 180 项具有挑战性的任务：SpreadsheetBench (Ma et al., 2024)、PPTC (Guo et al., 2023)，以及我们内部开发的 Writer 领域任务。我们根据需要对这些任务进行适配，使其能够集成到 OSWorld 框架中。得到的基准称为 **OfficeWorld**，可用于系统衡量智能体在办公场景中的能力。结果见表 2。

表 2: AutoGLM-OS 在 OfficeWorld 上与常用基线相比的性能。我们采用相同的框架（含工具）与测试设置，以保证比较公平。

智能体模型	Word	Excel	PPT	平均值
DeepSeek-V3.1 (Liu et al., 2024a)	6.7	35.0	21.7	21.1
DeepSeek-R1 Guo et al. (2025)	13.3	36.7	18.3	22.8
Claude 3.7 Sonnet (Anthropic, 2023)	15.0	25.0	25.0	21.7
Claude 4.0 Sonnet (Anthropic, 2023)	18.3	35.0	20.0	24.4
Gemini-2.5-Pro (Team et al., 2023)	5.0	11.7	20.0	12.2
GPT-4o (Hurst et al., 2024)	18.3	21.7	8.3	16.1
GPT-4.1 (Achiam et al., 2023)	21.7	25.0	28.3	25.0
OpenAI o3 (Jaech et al., 2024)	<u>23.3</u>	36.7	<u>41.7</u>	33.9
COMPUTERRL (本文)				
w/ GLM-4-9B-0414	21.7	58.3	43.3	<u>41.1</u>
w/ GLM-4.1V-9B-Thinking	30.0	58.3	<u>41.7</u>	43.3

4.3 消融研究

为了评估不同算法与训练数据集对智能体性能的影响，我们在表 3 中给出了 OSWorld 基准上的消融研究。

框架消融。我们使用 GPT-4o，对仅使用 GUI 的方法与所提出的 API-GUI 范式进行性能比较。结果表明，API-GUI 范式在所有领域都大幅优于仅使用 GUI 的基线。具体而言，

表 3: 框架设计与训练方法的消融研究。我们将 OSWorld 划分为五个不同领域，以便更细致地比较不同策略在各领域中的效果。

方法	OS	Office	Daily	Professional	Workflow	平均值
框架消融（使用 GPT-4o）						
仅 GUI	41.7	6.2	12.3	14.3	7.5	11.2
API-GUI	52.6	27.9	25.7	41.6	10.8	26.2
训练消融						
未训练	20.8	17.2	19.7	22.9	3.3	15.2
+ 行为克隆	54.2	35.0	37.2	45.8	10.8	31.9
+ RL 阶段 1	83.3	46.1	45.1	56.3	16.1	42.0
+ Entropulse	75.0	42.3	50.6	52.1	18.9	41.5
+ RL 阶段 2	83.3	46.2	46.7	60.4	27.2	45.8

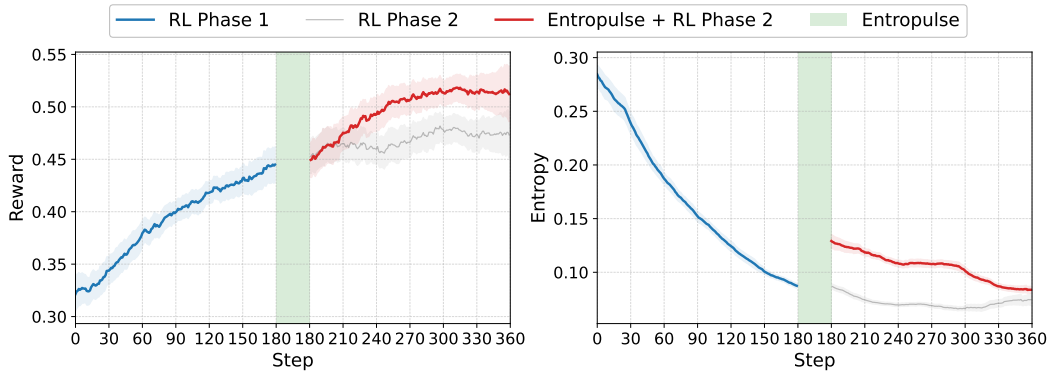


图 5: COMPUTERL 的训练奖励曲线（左）与熵曲线（右），阴影为 95% 置信区间。红线表示在第一阶段 RL 后通过 Entropulse 恢复熵的训练过程，灰线则表示仅重置参考模型后继续训练。

API-GUI 策略的平均成功率达到 26.2%，相较仅使用 GUI 的方法（11.2%）提升了 134%。最显著的增益出现在 Office 领域（27.9% 对 6.2%）和 Professional 领域（41.6% 对 14.3%），API-GUI 分别带来 350% 和 191% 的提升。这些结果验证了我们的核心假设：将 API 调用与 GUI 交互结合，可以更高效、更可靠地执行任务，尤其适用于能够受益于程序化控制的复杂专业工作流。

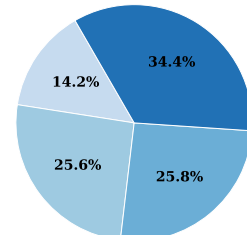
训练消融。我们使用 Qwen2.5-14B 研究不同训练阶段逐步产生的影响。从骨干模型出发，行为克隆（BC）以 31.9% 的性能奠定了坚实基础。第一阶段 RL（RL1）带来显著增益，使性能提升至 42.0%（提高 10.1 个百分点）。值得注意的是，Entropulse 阶段在保持近似性能（41.5%）的同时，显著提高了动作熵；这增强了探索多样性，使最终的 RL2 阶段能够进一步改进。RL2 阶段达到 45.8% 的最佳性能（相较 RL1 提高 3.8 个百分点），受益于 Entropulse 所引入的更强探索能力。值得一提的是，Workflow 领域在整个训练过程中提升最为显著（10.8% $\rightarrow$ 27.2%），其他领域则始终保持较高性能，这突出表明多阶段训练十分重要。

RL 可扩展性。我们在图 5 中给出 RL 的训练奖励与熵曲线，以研究 Entropulse 对延长 RL 训练动态的影响。第一阶段 RL 收敛后，我们比较采用与不采用 Entropulse 的第二阶段 RL。为保证比较公平，我们在两种设置中都重置参考模型。

结果表明，引入 Entropulse 会提高模型的熵，从而恢复其探索能力。增强后的探索显著扩展了有效训练步数，并最终提升整体性能。

4.4 案例研究与错误分析

我们在桌面环境中开展案例研究，以识别系统潜在的优化方向。尽管模型在大多数场景中表现稳健，我们仍发现了若干局限。具



体而言, 任务执行过程中遇到的错误可以分为四种主要类型: 视觉感知错误、多应用协调失败、操作幻觉以及其他错误。这些错误类型的分布见图 6。

附录 G 还给出了更多具有代表性的成功与失败案例, 以进一步说明模型的能力与局限。

5 相关工作

大语言模型。 GPT-4 (Achiam et al., 2023)、Gemini (Team et al., 2023)、Claude-3 (Anthropic, 2023)、Llama (Touvron et al., 2023a)、ChatGLM (Zeng et al., 2022; Du et al., 2022)、Qwen (Bai et al., 2023a; Team, 2024) 和 DeepSeek (Liu et al., 2024a; Guo et al., 2025) 等 LLM 在知识表示与自然语言理解方面展现了卓越能力, 推动了多样化的下游应用。视觉语言模型 (VLM) (Hong et al., 2023; 2025; Bai et al., 2023b; Hurst et al., 2024) 进一步将 LLM 扩展到多模态输入, 使模型能够对文本与图像进行联合推理, 以完成视觉问答等任务。

计算机使用智能体。 CogAgent (Hong et al., 2023) 引入了多模态 GUI 理解能力。AutoGLM (Liu et al., 2024c) 将规划与定位解耦, 并通过在线 RL 加以改进。OS-Atlas (Wu et al., 2024b) 提出一种基础 GUI 动作模型。Aguvis (Xu et al., 2024) 通过视觉训练实现跨平台交互。PC-Agent-E (He et al., 2025) 使用轨迹增强来提高熟练度。UI-TARS (Qin et al., 2025) 根据屏幕截图执行类人的 GUI 交互。Agent S2 (Agashe et al., 2025) 将定位与层次化推理结合。CUA (OpenAI, 2025) 提供可编程的桌面自动化能力。

计算机使用基准。 WebArena (Zhou et al., 2023) 为在线交互提供模拟网站, 但存在两方面局限: 与真实环境有差异, 而且只关注 Web。其他专注 Web 的基准 (Yao et al., 2022; Koh et al., 2024; Chezelles et al., 2024; Miyai et al., 2025) 也存在类似问题。软件工程基准 (Jimenez et al., 2023; Yang et al., 2024a; Li et al., 2024; Zan et al., 2025; Padigela et al., 2025) 则缺少全面的桌面评测。OSWorld (Xie et al., 2024) 通过 369 项任务与 134 个评估函数弥补了这些空白。Windows Agent Arena (Bonatti et al., 2024) 进一步扩展了该方向, 提供 150 多项基于 Windows 的任务。

面向 LLM 的 RL 训练。 PPO (Schulman et al., 2017) 解决了 RL 训练中策略梯度不稳定的问题。GRPO (Guo et al., 2025) 通过组采样扩展 PPO, 并移除了价值更新。DAPO (Yu et al., 2025) 对 GRPO 进行改进, 以提高稳定性。ARPO (Lu et al., 2025) 引入回放缓冲区与任务选择机制。ZeroGUI (Yang et al., 2025b) 利用 VLM 自动生成任务并估计奖励。Verl (Sheng et al., 2025) 提出用于动态调度的混合编程方法。AReaL (Fu et al., 2025) 通过将模型生成解耦来实现异步 RL。

6 结论

在本工作中, 我们提出 COMPUTERL, 这是一套新的计算机使用智能体框架, 通过整合基于 API 的动作与 GUI 动作、高效数据收集及可扩展 RL 训练, 推进桌面自动化。我们在 OSWorld 与 OfficeWorld 上的实验表明, 该框架在推理、准确率和泛化能力方面均优于既有方法。本工作为构建能力更强、运行更持久的自主计算机使用智能体奠定了基础, 并为智能人机交互的未来研究指明了具有潜力的方向。

REFERENCES

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Saaket Agashe, Kyle Wong, Vincent Tu, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent s2: A compositional generalist-specialist framework for computer use agents. *arXiv preprint arXiv:2504.00906*, 2025.
- Anthropic. Model card and evaluations for claude models. 2023.
- Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023a.
- Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-vl: A frontier large vision-language model with versatile abilities. *arXiv preprint arXiv:2308.12966*, 2023b.
- Michael Bain and Claude Sammut. A framework for behavioural cloning. In *Machine intelligence 15*, pp. 103–129, 1995.
- Rogério Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Buckner, et al. Windows agent arena: Evaluating multi-modal os agents at scale. *arXiv preprint arXiv:2409.08264*, 2024.
- Ivan Bratko, Tanja Urbančič, and Claude Sammut. Behavioural cloning: phenomena, results and problems. *IFAC Proceedings Volumes*, 28(21):143–149, 1995.
- De Chezelles, Thibault Le Sellier, Sahar Omid Shayegan, Lawrence Keunho Jang, Xing Han Lü, Ori Yoran, Dehan Kong, Frank F Xu, Siva Reddy, Quentin Cappart, et al. The browsergym ecosystem for web agent research. *arXiv preprint arXiv:2412.05467*, 2024.
- Zhengxiao Du, Yujie Qian, Xiao Liu, Ming Ding, Jiezhong Qiu, Zhilin Yang, and Jie Tang. Glm: General language model pretraining with autoregressive blank infilling. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 320–335, 2022.
- Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for llm agent training. *arXiv preprint arXiv:2505.10978*, 2025.
- Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, et al. Areal: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*, 2025.
- Team GLM, Aohan Zeng, Bin Xu, Bowen Wang, Chenhui Zhang, Da Yin, Diego Rojas, Guanyu Feng, Hanlin Zhao, Hanyu Lai, et al. Chatglm: A family of large language models from glm-130b to glm-4 all tools. *arXiv preprint arXiv:2406.12793*, 2024.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Yiduo Guo, Zekai Zhang, Yaobo Liang, Dongyan Zhao, and Nan Duan. Pptc benchmark: Evaluating large language models for powerpoint task completion. *arXiv preprint arXiv:2311.01767*, 2023.
- Yanheng He, Jiahe Jin, and Pengfei Liu. Efficient agent training for computer use. *arXiv preprint arXiv:2505.13909*, 2025.
- Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. Cogagent: A visual language model for gui agents. *arXiv preprint arXiv:2312.08914*, 2023.

-
- Wenyi Hong, Wenmeng Yu, Xiaotao Gu, Guo Wang, Guobing Gan, Haomiao Tang, Jiale Cheng, Ji Qi, Junhui Ji, Lihang Pan, et al. Glm-4.1 v-thinking: Towards versatile multimodal reasoning with scalable reinforcement learning. *arXiv e-prints*, pp. arXiv-2507, 2025.
- Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.
- Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.
- Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. *arXiv preprint arXiv:2401.13649*, 2024.
- Bin Lei, Yuchen Li, Yiming Zeng, Tao Ren, Yi Luo, Tianyu Shi, Zitian Gao, Zeyu Hu, Weitai Kang, and Qiuwu Chen. Infant agent: A tool-integrated, logic-driven agent with cost-effective api usage. *arXiv preprint arXiv:2411.01114*, 2024.
- Bowen Li, Wenhan Wu, Ziwei Tang, Lin Shi, John Yang, Jinyang Li, Shunyu Yao, Chen Qian, Binyuan Hui, Qicheng Zhang, et al. Devbench: A comprehensive benchmark for software development. *CoRR*, 2024.
- Kefan Li and Yuan Yuan. Large language models as test case generators: Performance evaluation and enhancement, 2024. URL <https://arxiv.org/abs/2404.13340>.
- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024a.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023.
- Xiao Liu, Bo Qin, Dongzhu Liang, Guang Dong, Hanyu Lai, Hanchen Zhang, Hanlin Zhao, Iat Long Iong, Jiadai Sun, Jiaqi Wang, Junjie Gao, Junjun Shan, Kangning Liu, Shudan Zhang, Shuntian Yao, Siyi Cheng, Wentao Yao, Wenyi Zhao, Xinghan Liu, Xinyi Liu, Xinying Chen, Xinyue Yang, Yang Yang, Yifan Xu, Yu Yang, Yujia Wang, Yulin Xu, Zehan Qi, Yuxiao Dong, and Jie Tang. Autoglm: Autonomous foundation agents for guis, 2024b. URL <https://arxiv.org/abs/2411.00820>.
- Xiao Liu, Bo Qin, Dongzhu Liang, Guang Dong, Hanyu Lai, Hanchen Zhang, Hanlin Zhao, Iat Long Iong, Jiadai Sun, Jiaqi Wang, et al. Autoglm: Autonomous foundation agents for guis. *arXiv preprint arXiv:2411.00820*, 2024c.
- Fanbin Lu, Zhisheng Zhong, Shu Liu, Chi-Wing Fu, and Jiaya Jia. Arpo: End-to-end policy optimization for gui agents with experience replay. *arXiv preprint arXiv:2505.16282*, 2025.
- Zeyao Ma, Bohan Zhang, Jing Zhang, Jifan Yu, Xiaokang Zhang, Xiaohan Zhang, Sijia Luo, Xi Wang, and Jie Tang. Spreadsheetbench: Towards challenging real world spreadsheet manipulation. *Advances in Neural Information Processing Systems*, 37:94871–94908, 2024.
- Atsuyuki Miyai, Zaiying Zhao, Kazuki Egashira, Atsuki Sato, Tatsumi Sunada, Shota Onohara, Hiromasa Yamanishi, Mashiro Toyooka, Kunato Nishina, Ryoma Maeda, et al. Webchorearena: Evaluating web browsing agents on realistic tedious web tasks. *arXiv preprint arXiv:2506.01952*, 2025.

-
- OpenAI. Computer-using agent: Introducing a universal interface for ai to interact with the digital world. 2025. URL <https://openai.com/index/computer-using-agent>.
- Harshith Padigela, Chintan Shah, and Dinkar Juyal. Ml-dev-bench: Comparative analysis of ai agents on ml development workflows. *arXiv preprint arXiv:2502.00964*, 2025.
- Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 1279–1297, 2025.
- Qiushi Sun, Kanzhi Cheng, Zichen Ding, Chuanyang Jin, Yian Wang, Fangzhi Xu, Zhenyu Wu, Chengyou Jia, Liheng Chen, Zhoumianze Liu, et al. Os-genesis: Automating gui agent trajectory construction via reverse task synthesis. *arXiv preprint arXiv:2412.19723*, 2024.
- Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- Qwen Team. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023a.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023b.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *arXiv preprint arXiv:2308.11432*, 2023.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. Os-copilot: Towards generalist computer agents with self-improvement. *arXiv preprint arXiv:2402.07456*, 2024a.
- Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, et al. Os-atlas: A foundation action model for generalist gui agents. *arXiv preprint arXiv:2410.23218*, 2024b.
- Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wensen Cheng, Qi Zhang, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, and Tao Gui. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.

-
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024.
- Tianbao Xie, Jiaqi Deng, Xiaochuan Li, Junlin Yang, Haoyuan Wu, Jixuan Chen, Wenjing Hu, Xinyuan Wang, Yuhui Xu, Zekun Wang, et al. Scaling computer-use grounding via user interface decomposition and synthesis. *arXiv preprint arXiv:2505.13227*, 2025.
- Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguis: Unified pure vision agents for autonomous gui interaction. *arXiv preprint arXiv:2412.04454*, 2024.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025a. URL <https://arxiv.org/abs/2505.09388>.
- Chenyu Yang, Shiqian Su, Shi Liu, Xuan Dong, Yue Yu, Weijie Su, Xuehui Wang, Zhaoyang Liu, Jinguo Zhu, Hao Li, et al. Zerogui: Automating online gui learning at zero human cost. *arXiv preprint arXiv:2505.23762*, 2025b.
- John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, et al. Swe-bench multimodal: Do ai systems generalize to visual software domains? *arXiv preprint arXiv:2410.03859*, 2024a.
- Yuhao Yang, Yue Wang, Dongxu Li, Ziyang Luo, Bei Chen, Chao Huang, and Junnan Li. Aria-ui: Visual grounding for gui instructions. *arXiv preprint arXiv:2412.16256*, 2024b.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757, 2022.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, et al. Multi-swe-bench: A multilingual benchmark for issue resolving. *arXiv preprint arXiv:2504.02605*, 2025.
- Aohan Zeng, Xiao Liu, Zhengxiao Du, Zihan Wang, Hanyu Lai, Ming Ding, Zhuoyi Yang, Yifan Xu, Wendi Zheng, Xiao Xia, et al. Glm-130b: An open bilingual pre-trained model. In *The Eleventh International Conference on Learning Representations*, 2022.
- Hanchen Zhang, Xiao Liu, Bowen Lv, Xueqiao Sun, Bohao Jing, Iat Long Iong, Zehan Qi, Hanyu Lai, Yifan Xu, Rui Lu, Zhenyu Hou, Hongning Wang, Jie Tang, and Yuxiao Dong. Agentrl: Reinforce all-round agents from zero. *arXiv preprint*, 2025.
- Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. In *Second Agent Learning in Open-Endedness Workshop*, 2023.

译者说明

本译本逐句翻译摘要及全部正文部分；为保留原论文的完整补充材料与交叉引用，以下附录沿用英文原文。

A API DEVELOPMENT WORKFLOW

In this section, we detail the methodology for leveraging LLMs to automate API construction. We propose a semi-automated workflow wherein users need only supply exemplar tasks performed within the target application; the LLM then autonomously generates both the necessary API code and corresponding test cases. The workflow comprises three primary stages: requirement analysis, API implementation, and test case generation.

Requirement Analysis During the requirements analysis phase, users provide a set of task examples related to the target application as input. The workflow leverages the LLM to analyze these task instances, extracting the essential functionalities required for task completion. It then compares these requirements against the existing API interface definitions to identify potential gaps. If uncovered functionalities are detected, the system automatically generates new API interfaces along with their corresponding parameter specifications.

Notably, we limit the generated interfaces to encapsulate only general-purpose functionalities, thereby avoiding excessive complexity and the proliferation of APIs. This design choice mitigates implementation difficulty and reduces the adaptation burden on the agent.

API Implementation Upon obtaining the interface definitions, the workflow systematically iterates over each interface and its associated parameters. For each specification, it leverages the designated Python libraries of the target application to implement the corresponding API functionalities. Additionally, the workflow incorporates error-handling mechanisms and logging to facilitate human debugging and maintenance. This automated approach not only streamlines API development but also enhances consistency and reusability across different application contexts.

Test Case Generation Following the implementation of API functionalities, the workflow conducts fundamental unit testing to ensure the correctness and robustness of each API. Specifically, the testing process verifies: (1) whether the API can be invoked without runtime errors, and (2) whether the API returns correct results across a range of parameter inputs. For API implementations that fail these tests, the workflow provides detailed error feedback to the API implementation module, which then autonomously attempts corrections until the APIs pass all tests.

This automated methodology substantially lowers the manual effort required, enabling the creation of application-specific API sets with minimal human intervention. As a result, the barrier for users to develop APIs for diverse applications is significantly reduced. We have developed API sets for multiple commonly used default applications in Ubuntu and integrated them into our Ubuntu virtual machine environment.

B ACTION SPACE

Our action space for AUTOGLM-OS is shown in Table 4, and our API number for each application is in Table 5.

表 4: Action space of AUTOGLM-OS

Function	Description
<code>open_app(app_name)</code>	Open specified application (e.g., Chrome, Terminal).
<code>click(coordinates, num_clicks, button_type)</code>	Click at coordinates $[x, y]$ with the specified mouse button and number of clicks.
<code>type(coordinates, text, overwrite, enter)</code>	Type text at coordinates; optionally overwrite existing content and/or press Enter.
<code>drag_and_drop(drag_from, drop_on)</code>	Drag from $[x_1, y_1]$ and drop onto $[x_2, y_2]$.
<code>scroll(coordinates, direction)</code>	Scroll at coordinates in direction (up / down).
<code>switch_window(window_id)</code>	Switch focus to the window with given ID.
<code>hotkey(keys)</code>	Press a key combination (e.g., <code>[ctrl, c]</code>).
<code>quote(content)</code>	Record content for memory.
<code>wait()</code>	Pause execution temporarily.
<code>exit(success)</code>	Terminate task with success (True) or failure (False).

表 5: Statistics of the number of available APIs per application

Application	Number of APIs
Code	12
Chrome	11
LibreOffice Calc	27
LibreOffice Impress	22
LibreOffice Writer	19
VLC	12
Total	103

C PROMPT FORMULATION FOR AUTOGLM-OS

The design of the observation space is pivotal, as it directly constrains the upper bound of the agent’s performance. In this section, we detail the integration of the API set with GUI operations, alongside the incorporation of contextual information from the desktop environment, to systematically construct both the agent’s observation space and action space. This unified framework ensures that the designed observation and action spaces capture the complexity of real-world tasks, providing a solid foundation for robust agent learning and generalization.

Action space formulation The integration of a large number of APIs with GUI operations, while ensuring effective agent interaction, remains a significant challenge. In practice, we mitigate this complexity by dynamically detecting the currently active application to infer potentially relevant APIs, thereby reducing the number of available APIs and lowering the agent’s adaptation overhead. Furthermore, we use Python classes and descriptive docstrings to delineate each operation type, ensuring they are clearly interpretable by most LLMs. This object-oriented strategy enhances the model’s understanding and precision in performing operations. These classes are provided to the agent via the system prompt, enabling interaction through Python function calls. This design facilitates rapid agent adaptation and efficient generalization of operations across diverse applications. Additionally, the agent’s output format is standardized in the system prompt, which encourages the agent to interleave reasoning and action execution. This approach promotes enhanced planning and reflective capabilities within the agent, thus improving its overall performance in complex task execution scenarios.

Observation formulation To facilitate the effective perception and manipulation of GUIs by the model, we leverage the Python Accessibility Toolkit Service Provider Interface (pyatspi) to extract comprehensive attributes of desktop elements systematically. Each GUI element encompasses the element’s semantic type, visible text content, precise screen coordinates, and spatial dimensions. This structured representation enables the LLM agent to parse, ground, and reason over the GUI in a manner analogous to human users.

We present the element format of the environment ally tree in our observation space as follows:

tag	text	position (center x & y)	size (w & h)
-----	------	-------------------------	--------------

The tag is the XML tag of the element, such as div or button. The text is the text content of the element, which can be empty for elements that do not have text. The position is represented by the center coordinates (x, y) of the element, and the size is represented by its width (w) and height (h).

For the multimodal model, the ally tree is removed from the input. Instead, we capture the GUI screenshot at a resolution of 1920×1080 pixels (1080p) and subsequently resize it to 1280×720 pixels (720p), which serves as the input representation of the desktop environment.

Beyond the extraction of individual GUI components, we augment the input space with rich contextual metadata to provide a holistic depiction of the agent’s operational environment. Specifically, we provide a comprehensive enumeration of open desktop windows, including their hierarchical relationships, as well as the name and additional information of the currently focused application. To promote consistent and adaptive behavior, we also deliver feedback from the most recent GUI action or tool call, which may include environmental status updates, confirmations, or error signals.

The app format of the observation space is as follows:

Window ID	App Name	Title
-----------	----------	-------

The Window ID is the unique identifier of the application window, App Name is the name of the application, and Title is the title of the application window.

History formulation Given the extensive length of GUI observations and the inherent constraints imposed by the model’s context window, it is necessary to efficiently manage the input history across multiple interaction rounds. For each interaction, we omit redundant and detailed interface information while preserving the complete sequence of the agent’s reasoning process, actions taken, and the corresponding operation feedback. This approach ensures the retention of the essential operational trajectory, thereby maximizing the informativeness of the historical context while maintaining compatibility with the model’s capacity limitations.

Collectively, the components above constitute the observation space and action space of our agent. This representation not only enhances the agent’s environmental cognition but also enables better strategies for long-horizon planning and reasoning. As a result, the agent is better equipped to execute complex, multi-step tasks across diverse applications in the desktop environment.

Below is our detailed prompt organization for AUTOGLM-OS:

You are an agent which follow my instruction and perform desktop computer tasks as instructed.

You have good knowledge of computer and good internet connection and assume your code will run on a computer for controlling the mouse and keyboard.

For each step, you will get an observation of the desktop by 1) screenshot; 2) current application name; 3) accessibility tree, which is based on AT-SPI library; 4) application info; 5) last action result.

You should first generate a plan for completing the task, confirm the previous results, reflect on the current status, then generate operations to complete the task in python-style pseudo code using the predefined functions.

Your output should STRICTLY follow the format:

```
<think>
{**YOUR-PLAN-AND-THINKING**}
</think>
```python
{**ONE-LINE-OF-CODE**}
```
```

You will be provided access to the following methods to interact with the UI:

1. class Agent, a grounding agent which provides basic action space to interact with desktop.
2. class {tool_class_name}, which provides tools to interact with the current application {app_name}.

Here are the defination of the classes:

```
```python
{class_content}
```
```

* Note:

- Your code should be wrapped in ```python``` , and your plan and thinking should be wrapped in <think></think>.
- Only ****ONE-LINE-OF-CODE**** at a time.
- Each code block is context independent, and variables from the previous round cannot be used in the next round.
- Do not put anything other than python code in ```python```.
- You ****can only use the above methods to interact with the UI****, do not invent new methods.
- Return with `Agent.exit(success=True)` immediately after the task is completed.
- If you think cannot complete the task, ****DO NOT keep repeating actions**, just return with `Agent.exit(success=False)`.

-
- The computer's environment is Linux, e.g., Desktop path is '/home/user/Desktop'
 - My computer's password is 'password', feel free to use it when you need sudo rights

****IMPORTANT**** You are asked to complete the following task: {instruction}

Below is our history and input prompt for AUTOGLM-OS:

```
<|user|>
**Environment State (Omitted)**

<|assistant|>
<think>
{round0_thinking}
</think>
```python
{round0_operation}
```

<|user|>
**Environment State (Omitted)**
Previous Action Result: {round0_operation_feedback}

<|assistant|>
<think>
{round1_thinking}
</think>
```python
{round1_operation}
```

<|user|>
**Environment State (Omitted)**
Previous Action Result: {round1_operation_feedback}

<|assistant|>
<think>
{round2_thinking}
</think>
```python
{round2_operation}
```
...

<|user|>
{screenshot_for_multimodal}
* Apps: {all_apps}

* Current App: {cur_window_id}

* Ally Tree: {ally_tree_for_text}

* App Info: {app_info}

* Previous Action Result: {operation_feedback}
```

D TRAINING DETAILS

During the behavior cloning stage, we construct approximately 8,000 tasks through manual annotation and data augmentation. We employ multiple advanced models to generate diverse samples for each task, and subsequently apply the eval function to filter out successful trajectories. This process yields a high-quality BC dataset comprising roughly 180,000 steps, which is then used for SFT training. We employ a 16-node computing cluster for fine-tuning, with a maximum learning rate set to 1×10^{-5} , a sequence length of 32,768 tokens, and a global batch size of 256, over a total of three training epochs.

In the RL stage, the key training hyperparameters are summarized in Table 6. We initially train the BC policy (using the 1-epoch checkpoint for diversity) for 180 steps, after which performance improvements began to plateau. At this point, we collect rollouts during RL, perform task-level random selection, and curate approximately 130,000 additional steps of data for Entropulse training. The hyperparameters in this phase are identical to those used in the BC stage, except for a reduced learning rate of 5×10^{-6} . RL training is then resumed until a total of 360 steps have been reached.

表 6: Training configuration for RL training of AUTOGLM-OS.

| Category | Parameter (Value) |
|--------------------------------|--|
| Data | |
| Task type | Multi-turn chat |
| Max prompt length | 63,488 tokens |
| Max response length | 2,048 tokens |
| Train batch size | 32 |
| Responses per prompt | 16 |
| Concurrency | 1024 |
| Shuffle | True |
| Seed | 42 |
| Actor | |
| Exchange size | 1×10^{10} |
| Gradient checkpointing | Enabled |
| Strategy | FSDP |
| FSDP offloading | Param + Optimizer |
| Sequence parallel size | 2 |
| Max tokens / GPU | 32,768 |
| Precision dtype | bfloat16 |
| Algorithm | |
| Advantage estimator | GRPO |
| Discount factor γ | 1.0 |
| GAE parameter λ | 1.0 |
| Use remove padding | True |
| Use dynamic bsz | True |
| Mini-batch size | 32,768 |
| Micro-batch size / GPU | 1 |
| Logprob micro-batch size / GPU | 1 |
| KL loss | Enabled (<i>low_var_kl</i>), coef = 0.0003 |
| Entropy coefficient | 0.0 |
| Clip ratio | 0.2 |
| Optimizer | |
| Actor learning rate | 1×10^{-6} |
| LR warmup steps ratio | 0.0 |
| Warmup style | constant |
| Gradient clipping | 1.0 |
| Save frequency | 25 |
| Rollout | |
| Enable chunked prefill | True |
| Max new tokens (generation) | 2,048 |
| Do sample | True |
| Sampling temperature | 0.8 |
| Max turns | 30 |
| GPU memory utilization | 0.7 |
| Pools rollout | 2 |
| Pools other | 6 |

E HUMAN ANNOTATION PROTOCOL

Our annotation process involves ten trained annotators with master’s degrees, who are recruited and compensated in compliance with local labor laws and regulations. Annotators are provided with clear written guidelines to ensure consistency and accuracy, as outlined in our annotation protocol (see Figures 7 and 8). All tasks are designed to avoid sensitive personal data, and all annotated content is in English with no identifying information. The process includes task expansion—transforming generalized instructions into explicit, executable tasks—and strict result verification to minimize errors. Quality control measures include verification passes and clear formatting rules to improve annotation reliability. No annotator is exposed to harmful, discriminatory, or unsafe content during the process, and all work adheres to the Code of Ethics regarding fairness, privacy, and legal compliance.

Annotation Guidelines

I. Project Background

Currently, when agents perform different application tasks, various execution issues may arise. To address this problem, the core of this annotation task is:

1. First, **expand the user's seed instruction** into more executable instructions.
2. Then, **conduct strict result verification** after each task execution to ensure the accuracy of the execution chain.

II. Data Description

Phase 1: Task Expansion

Fields to be completed (No need to execute in a virtual machine, only need to fill in):

1. Expanded Task

- Modify the generalized task description into an **independent, executable** task.
- **No pronominal references** allowed.
- All operations must explicitly specify that they are performed **on a file located under /home/user/work/** (the prefix `/home/user/work/` is fixed).
- Instructions must be as clear and executable as possible.
 - **Prohibited example:** "Help me find some good movies" (too vague).
- You may appropriately **reduce the difficulty** of the task, for example:
 - If the original task is "Collect data from 2013–2020," you may adjust it to "Collect data from 2013–2015" (simplify content preparation for Phase 2).

2. Notes – Provide guidance for Phase 2 (described in Chinese for convenience).

- Example:

"Prepare a `test.docx` file containing multiple sentences, where the first letter of each sentence is lowercase. Verify using `compare_with_golden(result_path, golden_path)`. The result passes if the first English letter of each sentence is capitalized, other characters remain unchanged, and the content is exactly the same as the golden file."
- You must specify:
 - a. **What type of file** to prepare
 - b. **The file content format**

图 7: Document for Human Annotation (1 / 2)

-
- c. **Which function** to use for verification
 - d. **Which fields** to check during verification

Phase 2: Virtual Machine Function Verification & Field Completion

Fields to be completed:

1. **Target File**
 - All files in the task that require model operations (unprocessed files).
 - For `compare_with_golden` (only supports `xlsx`, `docx`, `csv`, `txt`):
 - If the task involves other file types or functions, these can be omitted.
2. **Detection Target**
 - The name(s) of the file(s) that need to be compared.
3. **Golden File** (*only filled when the verification function is `compare_with_golden`*)
 - Name of the standard answer file used for comparison.
4. **check_list** (*required if the Notes or task description explicitly specifies formatting*):
 - Optional if content comparison is default.
 - Single/multiple choice:
 - `font` : Compare font name, color, size, bold, italic, etc.
 - `fill` : Compare cell fill color.
 - `alignment` : Compare alignment (e.g., center).
 - `para_format` : Compare paragraph formatting (alignment, line spacing).
 - `table` : Compare text in inserted tables (format comparison not currently supported).
5. **Files**
 - All files required for the task.
 - You must create your own file names (arbitrary), but the file names **must be consistent across all fields** (important).

III. Annotation Notes

1. **Focus on evaluation** — ensure the model's required operations are properly executed; avoid cases where detection passes without actual modifications.
2. **All data must be in English**, ensure logical correctness after translation.
3. For manually typed scenarios, verify multiple times to minimize error rate:
 - Especially check IDs and file paths.
 - OCR (from screenshots) is recommended for accuracy.
4. Report issues promptly in the annotation group to avoid unnecessary rework.
5. For color-related tasks, use **red, yellow, blue, green**; avoid visually similar colors.
6. For downloading plugins or apps — **assume they are already installed**.

图 8: Document for Human Annotation (2 / 2)

F FUTURE DIRECTION

While our advancements with COMPUTERRL mark a significant leap forward in intelligent desktop automation, we see this work as a foundation for a radical transformation in human-computer interaction. Unleashing the full potential of autonomous agents on the desktop frontier demands reimagining long-standing paradigms across several axes.

F.1 TOWARDS ROBUST PERFORMANCE

AUTOGLM-OS has demonstrated remarkable proficiency across a spectrum of desktop tasks. However, genuine universality requires transcending current boundaries in coverage and generalization. Real-world digital environments are characterized by continual flux and heterogeneity, encompassing unfamiliar applications, emergent workflows, and rare edge cases that lie beyond the scope of existing datasets. A next-generation agent must dynamically adapt to shifting GUIs, unpredictable pop-ups, and entirely novel interfaces. To this end, we are re-architecting data pipelines to facilitate exponential expansion in training diversity and pioneering infrastructure to distill knowledge from real-world user interactions at scale continuously.

F.2 BREAKTHROUGHS IN LONG-HORIZON AUTONOMY

Envisioning the autonomous desktop assistant as an always-available cognitive collaborator necessitates mastering sustained, long-duration workflows. While current solutions excel at bounded, atomic tasks, they fall short of orchestrating complex, multi-step objectives over extended horizons. Our ambition is to endow the agent with hierarchical planning capabilities, allowing it to reason, learn, and revise strategies dynamically across arbitrarily long task sequences. Realizing this vision will catalyze a paradigm shift: automating not just discrete operations, but entire workstreams and creative processes end-to-end, fundamentally reshaping the productivity landscape for the cloud-native era.

F.3 FOUNDATIONS FOR SAFE AND ALIGNED AUTONOMY

Autonomous control over desktop platforms raises profound questions about safety, trustworthiness, and user agency. The margin for error narrows dramatically when agents are empowered to modify files, access sensitive data, or execute unbounded actions. Mitigating these risks requires a rigorous and principled approach to safe behavior and alignment. Our roadmap includes architecting granular permissioning frameworks, embedding robust pre-action validation, and multi-stage approval protocols. Ultimately, we aim to establish safety standards and best practices that can serve as foundational infrastructure, not just for our agent but for the future ecosystem of intelligent digital collaborators.

G DEMONSTRATION

This section presents examples drawn from a variety of application scenarios, including the initial four positive cases and the final two negative ones. Each example illustrates our API-GUI operational paradigm, which addresses the diverse challenges and requirements that arise within different applications.

G.1 CREATE MONTHLY SALES SUMMARY TABLE

The targeted task to be executed is "Create a table with two headers ('Month' and 'Total') in a new sheet to show the total sales for all months." The actual execution steps can be summarized as follows:

- Step1: Switch to 'Sheet2'
- Step2: Set cell 'A1' as "Month"
- Step3: Set cell 'B1' as "Total"
- Step4: Get monthly sales data from 'Sheet1'
- Step5: Switch to 'Sheet1'
- Step6: Switch to 'Sheet2'
- Step7: Set month names in column A
- Step8: Set total formula for Jan in 'B2'
- Step9: Set total formula for Feb in 'B3'
- Step10: Set total formula for Mar in 'B4'
- Step11: Set total formula for Apr in 'B5'
- Step12: Set total formula for May in 'B6'
- Step13: Set total formula for Jun in 'B7'
- Step14: Exit after completion

As Figure 9,10 shows, a summary table is created successfully.

G.2 CONVERT UPPERCASE TEXT TO LOWERCASE IN DOCUMENT

The targeted task to be executed is "Convert all uppercase text to lowercase in the document for consistent text presentation." The actual execution steps can be summarized as follows:

- Step1: Use the text case conversion tool to change all text to lowercase.
- Step2: Click the formatting menu.
- Step3: Select the specific formatting option.
- Step4: Open the relevant submenu.
- Step5: Select all text in the document.
- Step6: Click the formatting menu again.
- Step7: Select the specific formatting option again.
- Step8: Open the relevant submenu again to apply the changes.
- Step9: Exit after finishing the operation.

As Figure 11 shows, all uppercase text has been successfully converted to lowercase, ensuring a uniform and polished document appearance.

G.3 GENERATE SYSTEM RESOURCE MONITORING REPORT

The targeted task to be executed is "Use the 'sar' command in the 'sysstat' toolkit to monitor system activity and output the results to 'System_Resources_Report.txt' under Desktop." The actual execution steps can be summarized as follows:

- Step1: Open the Terminal application.
- Step2: Type the command `sar 1 30 > /Desktop/System_Resources_Report.txt`.
- Step3: Execute `sudo apt install sysstat` to install the toolkit.
- Step4: Enter the password if prompted.
- Step5: Re-type the sar command to start monitoring.
- Step6: Wait for 30 seconds while system activity is being recorded.
- Step7: Open the Files application.
- Step8: Navigate to and click on the report file under Desktop.
- Step9: Exit after confirming the report has been generated.

As Figure 12 shows, a system resource report file was generated successfully on the Desktop after monitoring, fulfilling the target task.

G.4 COPY IMAGE FROM .XCF FILE TO LIBREOFFICE WRITER AND SAVE

The targeted task to be executed is "Copy the image from an .xcf file located on the Desktop and paste it into a LibreOffice Writer document, then save the document as image.docx on the Desktop." The actual execution steps can be summarized as follows:

- Step1: Open the Files application.
- Step2: Navigate to the Desktop folder.
- Step3: Double-click to open the .xcf image file.
- Step4: Copy the image in the image editor.
- Step5: Open LibreOffice Writer.
- Step6: Paste the copied image into the Writer document.
- Step7: Save the document.
- Step8: Enter the filename as image.docx and select the Desktop as the save location.
- Step9: Navigate to the Desktop folder.
- Step10: Confirm and click save.

As Figure 13 shows, the image from the .xcf file was successfully pasted into a LibreOffice Writer document and saved as image.docx on the Desktop, completing the task.

G.5 QUESTION MISUNDERSTANDING ERROR

The targeted task to be executed is "Append `<br/>` to the end of each line in '1\n2\n3' and save in output.txt." The actual execution steps can be summarized as follows:

- Step1: Type the command `echo -e '1\n2\n3'` in the terminal.
- Step2: Exit after execution.

As Figure 14 shows, the agent misunderstood the requirement and only echoed the content without appending `<br/>` to each line or saving it into output.txt. This results in a task failure due to a misunderstanding of the question.

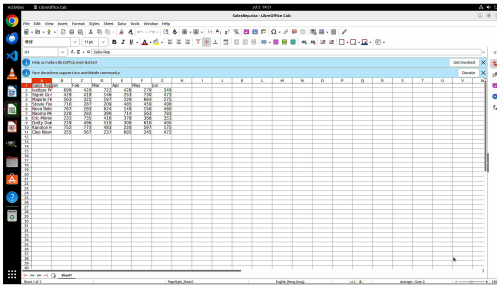
G.6 CLICK OPERATION ERROR

The targeted task to be executed is "Please help change GIMP's theme from dark to light."
The actual execution steps can be summarized as follows:

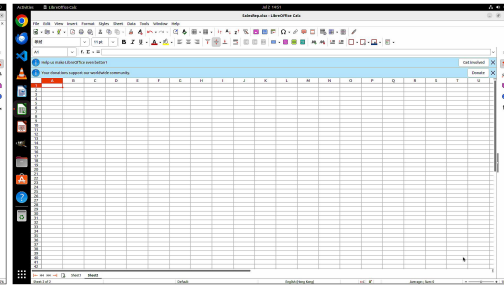
- Step1: Click on the menu in GIMP.
- Step2: Attempt to click the Preferences option.
- Step3: Repeat the click on Preferences.
- Step4: Try to use the shortcut Shift+Ctrl+P to open Preferences.
- Step5: Exit without successfully changing the theme.

As Figure 15 shows, the theme remains dark, indicating that the agent failed to change GIMP's theme to light due to incorrect click operations.

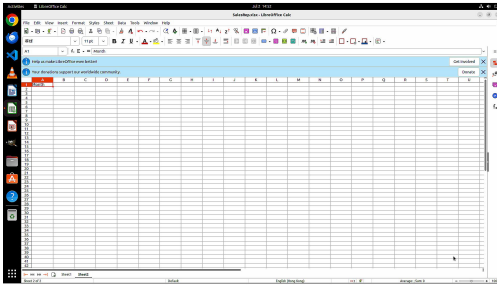
Step 1: CalcTools.switch_active_sheet('Sheet2')



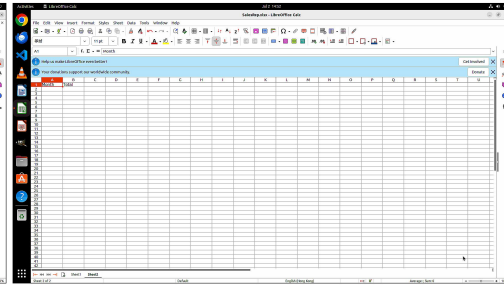
Step 2: CalcTools.set_cell_value('A1', 'Month')



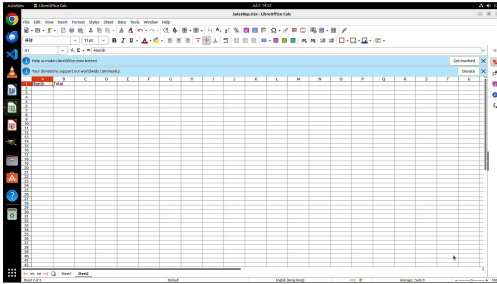
Step 3: CalcTools.set_cell_value('B1', 'Total')



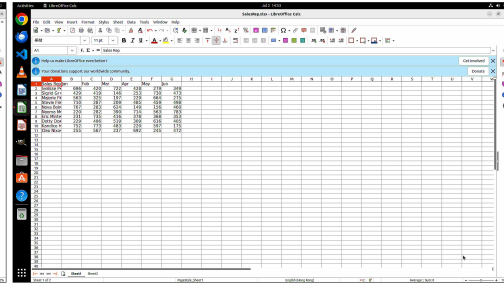
Step 4: CalcTools.get_column_data('B')



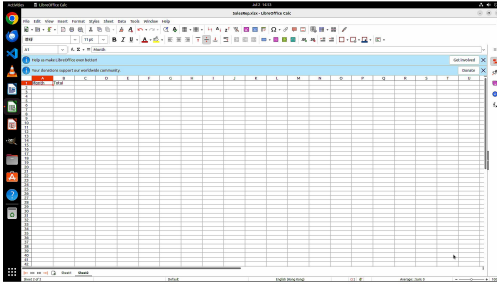
Step 5: CalcTools.switch_active_sheet('Sheet1')



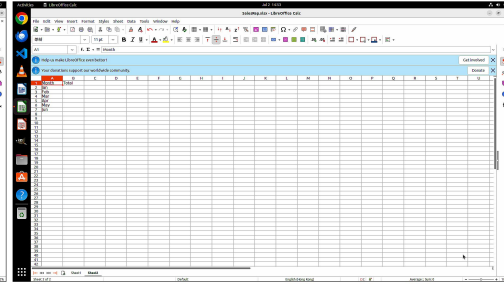
Step 6: CalcTools.switch_active_sheet('Sheet2')



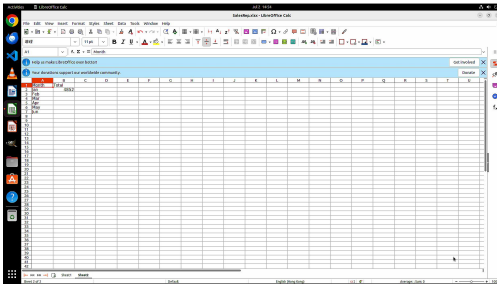
Step 7: CalcTools.set_column_values('A', ['Jan', 'Feb', ...])



Step 8: CalcTools.set_cell_value('B2', '=SUM(Sheet1.B2:...)')



Step 9: CalcTools.set_cell_value('B3', '=SUM(Sheet1.C2:...)')



Step 10: CalcTools.set_cell_value('B4', '=SUM(Sheet1.D2:...)')

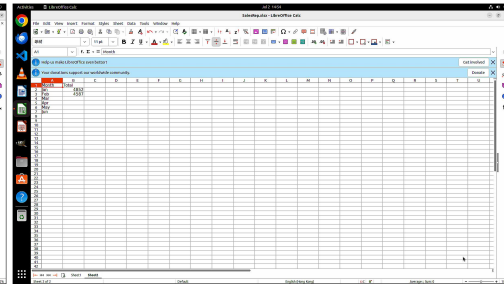


图 9: Task (Step 1-10): Create a table with two headers ("Month" and "Total") in a new sheet to show the total sales for all months.

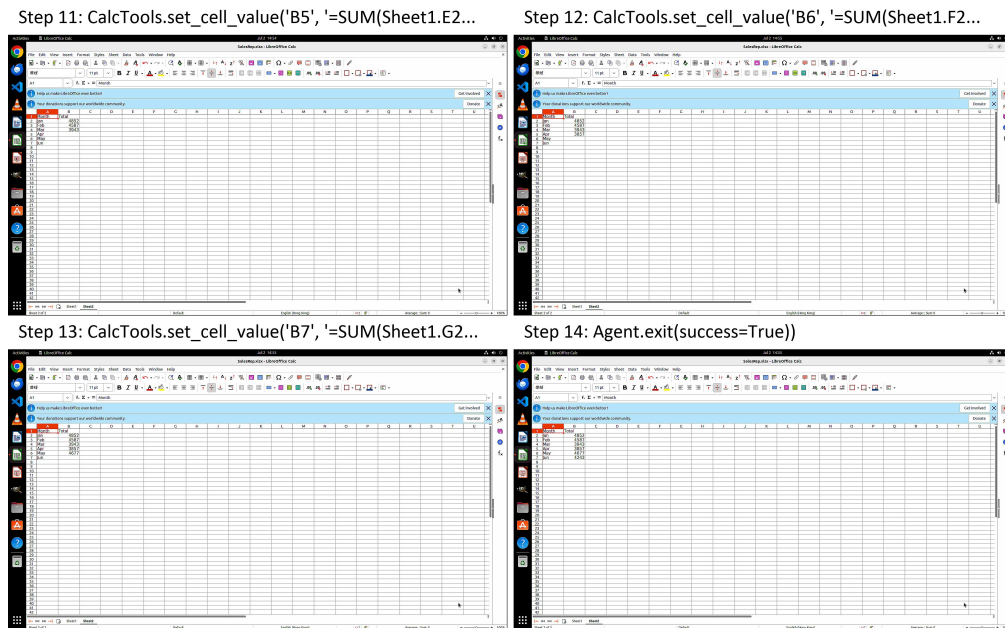
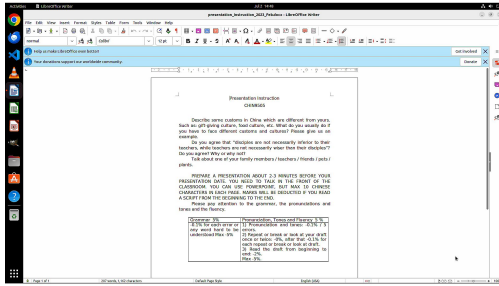
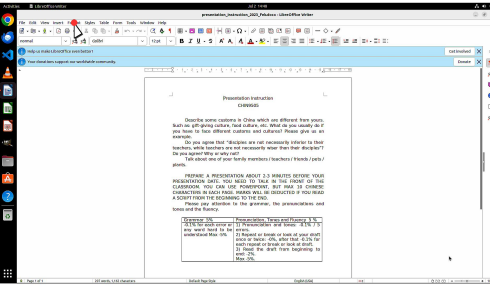


图 10: Task (Step 11-14): Create a table with two headers ("Month" and "Total") in a new sheet to show the total sales for all months.

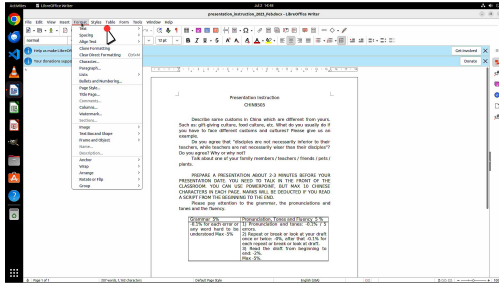
Step 1: `WriterTools.change_text_case('lowercase', '.*', ...`



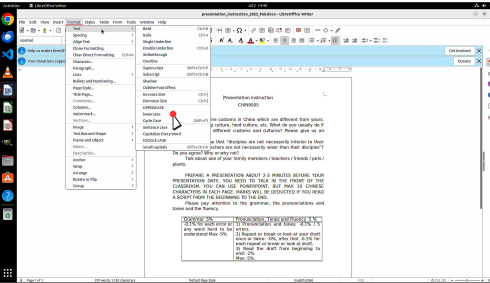
Step 2: `Agent.click([290, 76])`



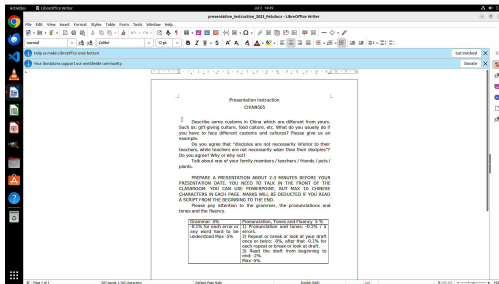
Step 3: `Agent.click([392, 101])`



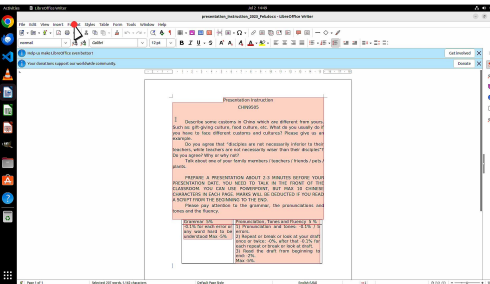
Step 4: `Agent.click([670, 430])`



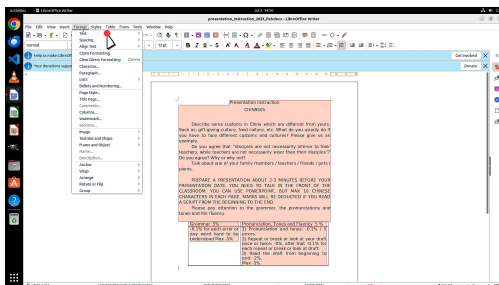
Step 5: `Agent.hotkey(['ctrl', 'a'])`



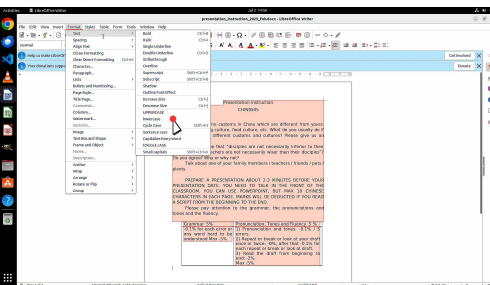
Step 6: `Agent.click([290, 76])`



Step 7: `Agent.click([392, 101])`



Step 8: `Agent.click([670, 430])`



Step 9: `Agent.exit(success=True)`

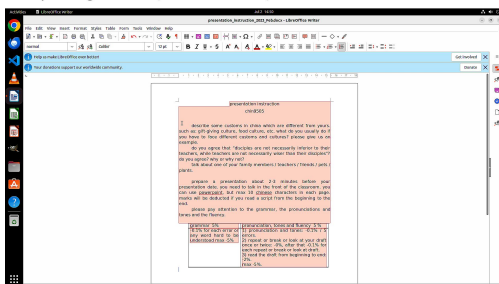
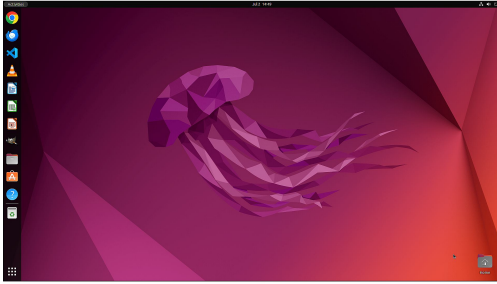
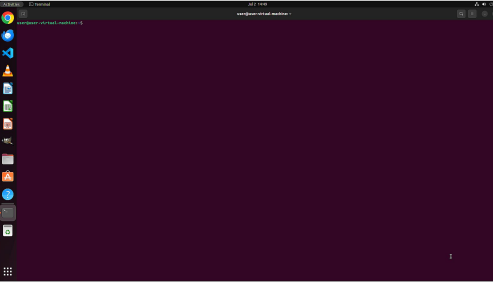


图 11: Task: I am currently engaged in text processing and require assistance in converting all uppercase text to lowercase within my document. This precision is critical for maintaining a uniform and polished presentation. Could you help me on this?

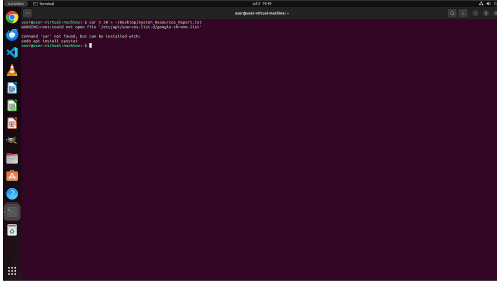
Step 1: Agent.open_app('terminal')



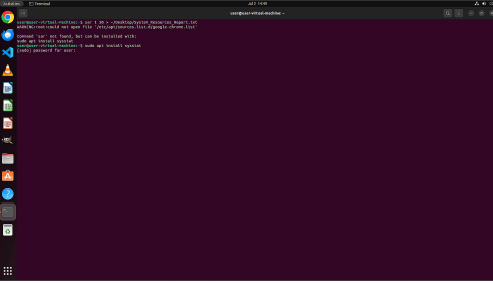
Step 2: Agent.type(text='sar 1 30 > ~/Desktop/System_Re...



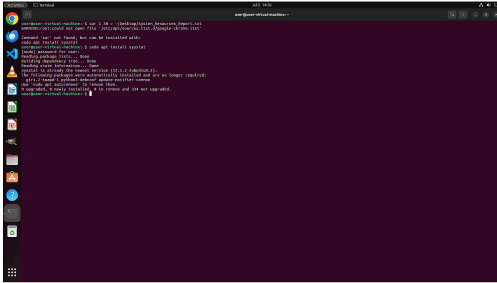
Step 3: Agent.type(text='sudo apt install sysstat', enter=...



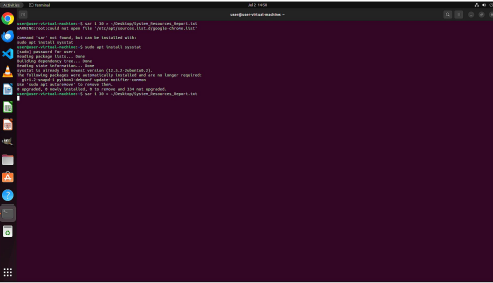
Step 4: Agent.type(text='password', enter=True)



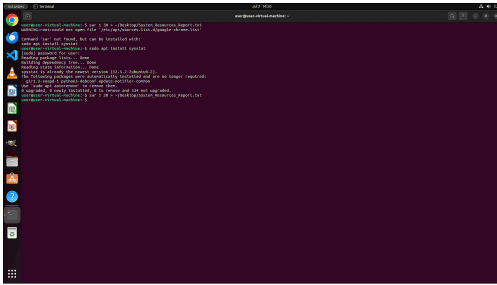
Step 5: Agent.type(text='sar 1 30 > ~/Desktop/System_Re...



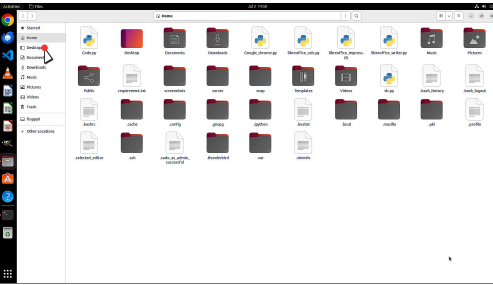
Step 6: Agent.wait()



Step 7: Agent.open_app('files')



Step 8: Agent.click([176, 172])



Step 9: Agent.exit(success=True)

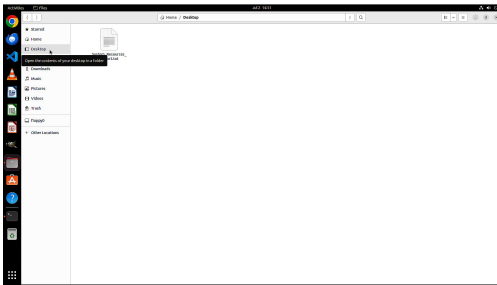
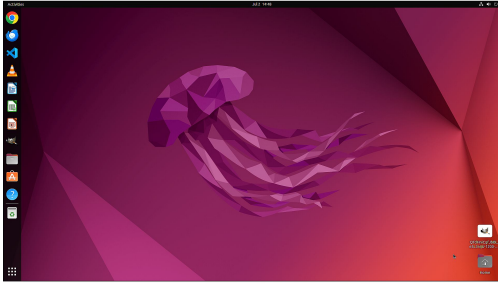
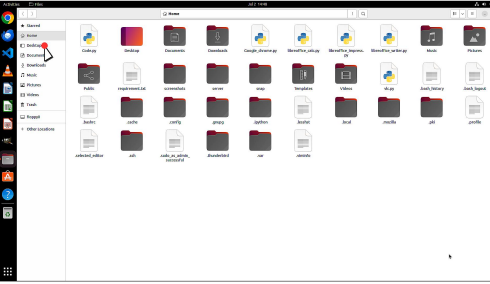


图 12: Task: Please use the 'sar' command in the 'sysstat' toolkit to monitor system activity, evaluate the status once every second for 30 seconds, output the results to "System_Resources_Report.txt" under Desktop.

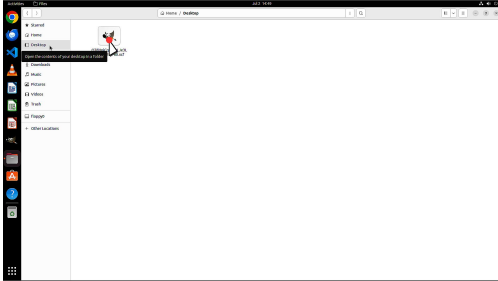
Step 1: Agent.open_app('files')



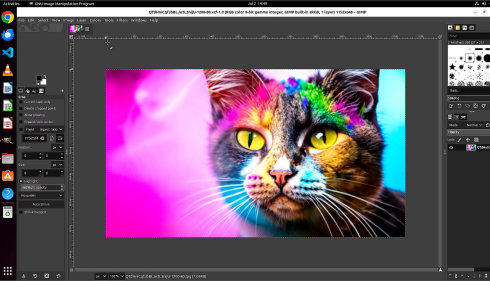
Step 2: Agent.click([176, 172])



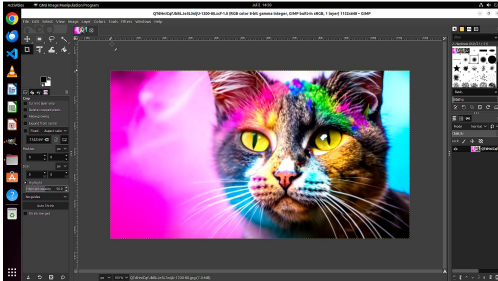
Step 3: Agent.click([408, 153], num_clicks=2)



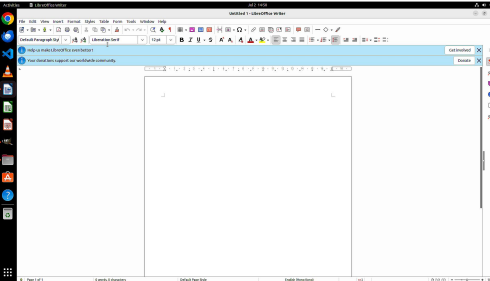
Step 4: Agent.hotkey(['ctrl', 'c'])



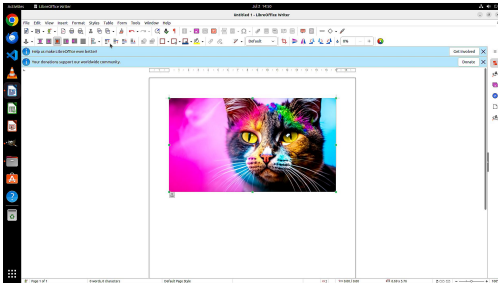
Step 5: Agent.open_app('libreoffice writer')



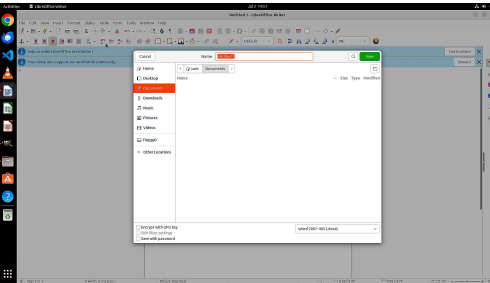
Step 6: Agent.hotkey(['ctrl', 'v'])



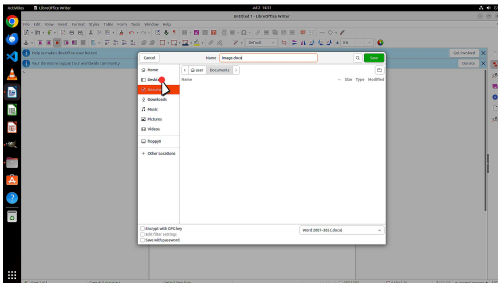
Step 7: Agent.hotkey(['ctrl', 's'])



Step 8: Agent.type([1021, 206], text='image.docx', over...



Step 9: Agent.click([610, 290])



Step 10: Agent.click([1427, 206])

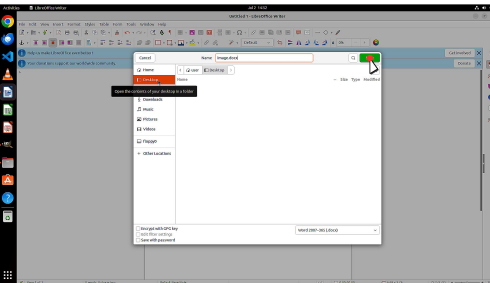
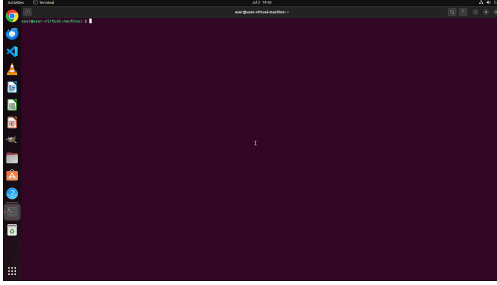


图 13: Task: I've stored my .xcf file on the Desktop. Can you assist me in copying the image and pasting it into a LibreOffice Writer document? Save the document as 'image.docx' on the Desktop, please.

Step 1: Agent.type(text="echo -e '1\\2\\3' | sed 's/\$/
'")



Step 2: Agent.exit(success=True)

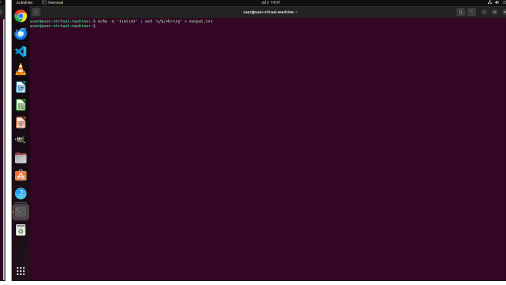
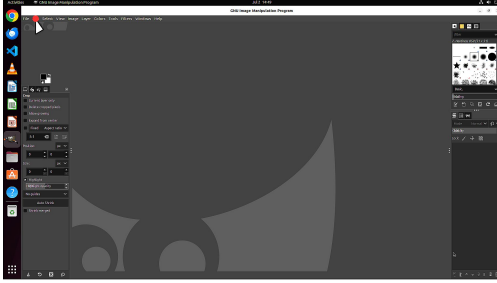
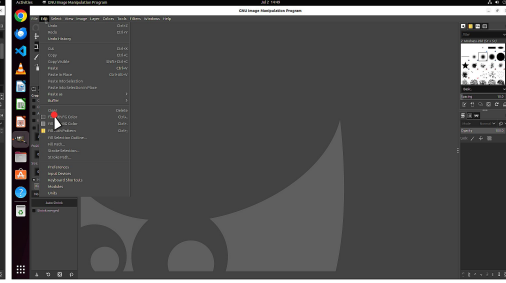


图 14: Fail Task (Question Misunderstanding Error): Append "
" to the end of each line in "1\\n2\\n3" and save in output.txt

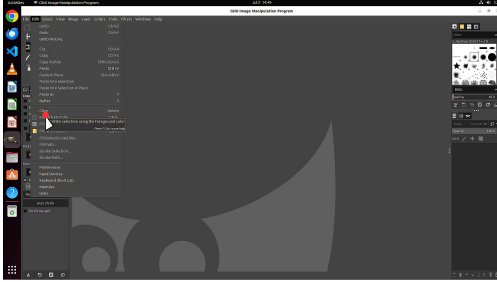
Step 1: Agent.click([125, 76])



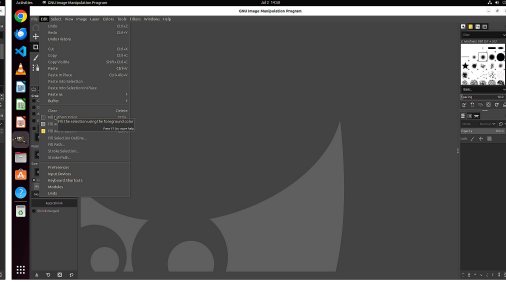
Step 2: Agent.click([163, 446])



Step 3: Agent.click([163, 446])



Step 4: Agent.hotkey(['shift', 'ctrl', 'p'])



Step 5: Agent.exit(success=False)

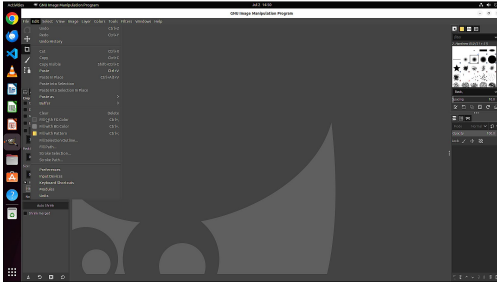


图 15: Fail Task (Click Operation Error): Please help change GIMP's theme from dark to light.